

A Mini Project Report on

LOG ANALYSIS USING ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING.

Submitted in partial fulfilment of requirement for the award of the degree of

MASTER OF TECHNOLOGY

In

CYBERSECURITY

Submitted By

SYED MOHAMMED MIFTAHUDDIN MARIF

100525742513

SANGI RAMYA

100525742511

Under the Guidance of

Prof. DR. K. SHYAMALA

Department Of CSE, UCEOU



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY COLLEGE OF ENGINEERING (AUTONOMOUS)
OSMANIA UNIVERSITY, HYDERABAD

UNIVERSITY COLLEGE OF ENGINEERING (AUTONOMOUS)

OSMANIA UNIVERSITY, HYDERABAD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the Seminar report entitled “**Log Analysis using Artificial Intelligence and Machine Learning**” is a bonafide work submitted by **Syed Mohammed Miftahuddin Marif** and **Sangi Ramya** in partial fulfilment for the requirements of the award of the degree in **MASTER OF TECHNOLOGY IN CYBERSECURITY** from University College of Engineering, Osmania University, Hyderabad, during the year **2025–2026**.

Project Guide

Prof. DR. K. SHYAMALA
Osmania University

Head Of the Department,CSE

Prof. Dr. K. SHYAMALA
Osmania University

UNIVERSITY COLLEGE OF ENGINEERING (AUTONOMOUS)
OSMANIA UNIVERSITY, HYDERABAD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

We, **Syed Mohammed Miftahuddin Marif**, bearing Roll Number **100525742513**, and **Sangi Ramya**, bearing Roll Number **100525742511**, hereby declare that the results presented in the project entitled: "LOG ANALYSIS USING ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING" are done and carried out by us during the year **2026** under the guidance of **Prof. DR. K. SHYAMALA** **Department of Computer Science and Engineering, University College of Engineering, Osmania University**, submitted in partial fulfilment of the requirements for the award of the degree of **Master of Technology in Cybersecurity**

This is a record of bona fide work carried out by us at the University College of Engineering, Osmania University, and the results embodied in this report have been reproduced or copied from any source. The results embodied in this report have not been submitted to any other University or Institution for the award of any degree or diploma.

DATE: 20/08/2026

PLACE: Hyderabad

Syed Mohammed Miftahuddin Marif

(100525742513)

Sangi Ramya

(100525742511)

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose constant guidance and encouragement crown all efforts with success.

We would like to thank our Principal **Prof. P. CHANDRA SEKHAR** and the management for providing all the facilities required for completing the project. We wish to express our profound thanks to **DR. K. SHYAMALA**, Head of the Computer Science and Engineering Department, for unflinching devotion, which led to the completion of our work.

We thank our project guide **Prof. DR. K. SHYAMALA**, Department of Computer Science and Engineering, for his valuable suggestions and encouragement throughout the work.

Last but not least, we express our heartfelt thanks to all our staff, parents, and classmates for their constant support, encouragement, and valuable contributions to the completion of the project.

SYED MOHAMMED MIFTAHUDDIN MARIF
100525742513

SANGI RAMYA
100525742511

Abstract

Secure Shell (SSH) servers exposed to the public internet are under near-constant probing by automated brute-force and credential-stuffing tools, generating log volumes that quickly outpace manual review. This paper presents the design, implementation, and evaluation of a hybrid intrusion detection system that combines unsupervised machine learning with an interpretable rule-based layer to identify malicious SSH activity directly from server logs. The system engineers behavioral features — failed password counts, distinct usernames attempted, authentication failure rates, and reverse-DNS mismatch signals — over sliding five-minute windows, grouped by source IP address. An Isolation Forest model, trained without labeled attack data, scores each window for statistical anomaly, while a parallel rule engine flags windows that match known attack signatures such as brute-force password guessing and username enumeration. The two signals are fused into a single ranked, severity-tagged alert list. The system was implemented end-to-end — from a offline batch detector to a real-time monitoring service with a Flask-based REST API and a live-polling web dashboard — and evaluated on the OpenSSH LogHub-2k benchmark dataset, a widely used collection of 2,000 real SSH server log lines spanning approximately four hours of activity. The system raised 21 ranked alerts covering 13 of 30 unique source IP addresses, correctly isolating a small number of high-volume brute-force sources — including one IP responsible for 141 failed password attempts in a single five-minute window — while requiring no manually labeled training data. This paper documents the complete system: its motivation, related work, dataset, architecture, implementation, experimental results, and limitations, and outlines directions for future work including raw-syslog ingestion and multi-host correlation.

Keywords: *intrusion detection, log analysis, anomaly detection, Isolation Forest, unsupervised machine learning, SSH brute-force detection, security information and event management, real-time monitoring*

Table of Contents

Abstract	2
1. Introduction	5
1.1 Background and Motivation	5
1.2 Problem Statement	5
1.3 Research Objectives	5
1.4 Contributions	6
1.5 Paper Organization	6
2. Related Work	7
2.1 Rule-Based and Signature-Based Detection	7
2.2 Automated Log Parsing	7
2.3 Supervised Machine Learning Approaches	7
2.4 Deep Learning and Sequence-Based Models	8
2.5 Unsupervised Anomaly Detection	8
2.6 Console and System Log Mining	8
2.7 Summary of the Research Gap	8
3. Dataset	10
3.1 Overview of OpenSSH LogHub-2k	10
3.2 Data Schema	10
3.3 Event Template Distribution	11
3.4 Descriptive Statistics	11
4. System Architecture and Methodology	13
4.1 Design Principles	13
4.2 High-Level Architecture	13
4.3 Log Parsing and Source IP Extraction	14
4.4 Feature Engineering	15
4.5 Unsupervised Anomaly Detection with Isolation Forest	15
4.6 Rule-Based Detection Layer	16
4.7 Alert Fusion and Severity Ranking	17
4.8 Real-Time Monitoring Architecture	17
5. Implementation	18
5.1 Technology Stack	18
5.2 Detection Engine (ssh_log_anomaly_detector.py)	18
5.3 Real-Time Server (realtime_server.py)	18
5.4 Log Simulator	19
5.5 Interactive Dashboard	19
5.6 Reproducibility	19
6. Experimental Results	20
6.1 Experimental Setup	20
6.2 Overall Detection Results	20
6.3 Temporal Pattern of Alerts	21

6.4 Case Studies of Detected Attacks	22
6.5 Real-Time System Behavior	22
6.6 Summary of Findings.....	22
7. Discussion	24
7.1 Effectiveness of the Hybrid Approach.....	24
7.2 Interpretability as a First-Class Requirement.....	24
7.3 Limitations	24
7.4 Comparison with Existing Approaches, Revisited	25
8. Conclusion and Future Work	25
8.1 Conclusion.....	25
8.2 Future Work.....	25
References.....	27
Appendix A: Configuration Parameters	28
Appendix B: REST API Reference	29
Appendix C: Complete Alert List.....	30

1. Introduction

1.1 Background and Motivation

Server logs are the primary evidentiary record of what actually happens on a running system. For internet-facing services, and Secure Shell (SSH) daemons in particular, this record is dominated not by legitimate use but by continuous, automated probing: botnets and scripted attackers scan public IP ranges around the clock, attempting common usernames and password combinations in the hope of finding a weakly configured host. A single production server can generate thousands of authentication-related log lines in a short window, the overwhelming majority of which are noise — rejected connections, invalid usernames, and failed passwords — punctuated rarely by a genuine, authorized login.

This creates a fundamental asymmetry that traditional log review cannot resolve: the volume of events scales with attacker activity, not with analyst capacity. Static, signature-based rules can catch known patterns but require continual manual maintenance and are blind to anything novel. Manual review does not scale past a handful of hosts. What is needed is a detection approach that can learn the shape of normal behavior directly from the data, flag what deviates from it, and do so without requiring a labeled dataset of “confirmed attacks” that, in practice, is rarely available for any given deployment.

1.2 Problem Statement

Given a stream of unstructured or semi-structured SSH server log lines, the problem addressed in this work is threefold: (1) extract meaningful behavioral signals from raw, per-line log events; (2) determine, without labeled training examples, which patterns of activity represent a plausible intrusion attempt; and (3) surface this determination as a ranked, interpretable alert that a human analyst can act on — not merely a numeric anomaly score. A fourth, practical requirement follows directly from how these systems are actually operated: detection must be usable in real time, against a continuously growing log, rather than only as an offline forensic exercise run after the fact.

1.3 Research Objectives

This work sets out to:

- Design a feature-engineering scheme that converts raw SSH log lines into behavioral, per-source-IP measurements over fixed time windows.
- Evaluate unsupervised anomaly detection, specifically Isolation Forest, as a mechanism for scoring those windows without labeled attack data.
- Complement the model with an interpretable rule layer that encodes known attack signatures (brute force, username enumeration, reverse-DNS spoofing) as explicit, auditable thresholds.
- Fuse both signals into a single ranked alert list with human-readable evidence, and evaluate the fused system on a public benchmark dataset.

- Extend the detector into a real-time service capable of monitoring a live, growing log file and exposing results through an API and an interactive dashboard.

1.4 Contributions

The primary contributions of this paper are:

1. A complete, reproducible hybrid detection pipeline combining unsupervised machine learning and rule-based detection for SSH log analysis, implemented in Python with scikit-learn and pandas.
2. A sliding-window, per-source-IP feature engineering scheme suited to behavioral (rather than per-line) anomaly detection in authentication logs.
3. An end-to-end evaluation on the OpenSSH LogHub-2k benchmark, including case studies of the specific attack sources the system identified.
4. A real-time extension of the offline detector — a file-watching service, REST API, and live-polling dashboard — demonstrating that the same detection logic can operate continuously rather than only in batch mode.

1.5 Paper Organization

The remainder of this paper is organized as follows. Section 2 reviews related work in rule-based intrusion detection, log parsing, and both supervised and unsupervised machine learning approaches to log anomaly detection. Section 3 describes the OpenSSH LogHub-2k dataset used for evaluation. Section 4 presents the proposed system architecture and methodology in detail. Section 5 describes the implementation. Section 6 reports experimental results. Section 7 discusses the strengths, limitations, and implications of the approach. Section 8 concludes and outlines future work.

2. Related Work

Log-based intrusion and anomaly detection has been studied from several distinct angles: signature-based detection systems, automated log parsing, supervised machine learning classifiers, and, more recently, deep sequence models. Each strand informs the design choices made in this work; each also carries limitations that motivate the hybrid approach proposed here.

2.1 Rule-Based and Signature-Based Detection

The dominant approach in operational Security Information and Event Management (SIEM) systems remains signature matching: an analyst or vendor encodes a known bad pattern — a specific sequence of failed logins, a known malicious IP range, a particular error string — as an explicit rule, and the system raises an alert whenever that pattern recurs. This approach is fast, cheap to evaluate, and highly interpretable: an analyst reading the alert knows exactly why it fired. Its central weakness is equally well documented — signature-based systems are structurally blind to any attack pattern that has not already been observed and encoded, and they require continual manual maintenance as attacker behavior evolves (Chandola, Banerjee, & Kumar, 2009).

2.2 Automated Log Parsing

Raw log messages are free text, but the overwhelming majority are generated from a small number of underlying source-code logging statements, differing only in their variable parameters (timestamps, usernames, IP addresses). A substantial body of work addresses the problem of automatically recovering this latent template structure. Drain (He, Zhu, Zheng, & Lyu, 2017) uses a fixed-depth parse tree to group log lines into templates online, without requiring a training pass over the full log corpus; earlier approaches such as Spell (Du & Li, 2016) apply longest-common-subsequence matching, and IPLoM (Makanju, Zincir-Heywood, & Milios, 2009) partitions logs iteratively by token count and position. Zhu et al. (2019) benchmark a wide range of these parsers and release the LogHub collection of real-world, hand-labeled log datasets — including the OpenSSH dataset used in this work — as a common evaluation ground. Log parsing research answers a necessary but different question than intrusion detection: it recovers structure from text, but does not by itself decide what structure is anomalous. The dataset used in this work arrives already parsed into templates (EventId / EventTemplate columns), which allowed this project to focus on the detection layer built on top of that structure.

2.3 Supervised Machine Learning Approaches

A second line of work treats log-based intrusion detection as a supervised classification problem: a model is trained on logs explicitly labeled as “normal” or “attack,” and learns a decision boundary between the two classes. When high-quality labels are available, supervised approaches can achieve strong precision and recall. In practice, however, labeled attack data at the scale needed for supervised training is rarely available for a given deployment — organizations do not typically maintain a curated, line-by-line labeled corpus of their own historical attacks, and labels from one environment often transfer poorly to another due to differing traffic patterns, usernames, and network topology

(Landauer, Onder, Skopik, & Wurzenberger, 2023). This label-scarcity problem is a central motivation for the unsupervised approach adopted in this paper.

2.4 Deep Learning and Sequence-Based Models

More recent work applies deep sequence models to log anomaly detection. DeepLog (Du, Li, Zheng, & Srikumar, 2017) models normal log sequences with a Long Short-Term Memory (LSTM) network and flags deviations from the learned execution pattern. LogBERT (Guo, Yuan, Wu, & Wu, 2021) applies a Transformer-based masked-token training objective to learn representations of normal log sequences, and flags sequences that the model finds difficult to reconstruct. These approaches can capture subtle sequential dependencies that simple count-based features miss, but they carry real operational costs: they typically require substantially more training data and compute than classical models, their internal decision process is difficult for a security analyst to inspect or trust at alert time, and they are frequently developed and evaluated in offline, batch settings rather than under the latency and resource constraints of a live monitoring service (Landauer et al., 2023).

2.5 Unsupervised Anomaly Detection

Isolation Forest (Liu, Ting, & Zhou, 2008) takes a different approach to unsupervised anomaly detection than density- or distance-based methods such as Local Outlier Factor. Rather than modeling what “normal” looks like directly, it exploits the fact that anomalous points are, by construction, few and different: a random partitioning of the feature space isolates an outlier in far fewer splits than it takes to isolate a point embedded in a dense cluster of similar points. This gives Isolation Forest three properties that make it well-suited to the problem addressed in this paper: it requires no labeled training data, it scales close to linearly with dataset size (Liu et al., 2008), and it operates directly on tabular, engineered features rather than requiring sequence or graph structure. General surveys of anomaly detection (Chandola et al., 2009) and of log-specific deep learning methods (Landauer et al., 2023) both note that classical tree-based ensemble methods remain competitive baselines precisely because of this simplicity and interpretability advantage, especially in settings — like this one — with limited or no labeled data.

2.6 Console and System Log Mining

Beyond security-specific work, broader console-log mining research established many of the foundational techniques this field relies on. Xu, Huang, Fox, Patterson, and Jordan (2009) demonstrated that source-code-derived log structure could be mined to detect large-scale system problems using Principal Component Analysis over per-request feature vectors — an early example of the same underlying idea used in this paper: convert unstructured log text into structured, countable features, and apply a general-purpose statistical or machine learning method on top.

2.7 Summary of the Research Gap

Table 1 summarizes the four families of prior approaches reviewed above against the practical requirements of this project: operating without labeled attack data, producing interpretable output, and being suitable for real-time deployment.

Approach	Needs Labels?	Interpretable?	Real-Time Suited?	Primary Limitation
Rule-based SIEM	No	Yes	Yes	Blind to novel/unseen attack patterns
Log parsing (Drain, Spell, IPLoM)	No	N/A (parsing only)	Yes	Recovers structure; does not itself detect anomalies
Supervised ML classifiers	Yes — large corpus	Moderate	Yes	Labeled attack data rarely available in practice
Deep sequence models (DeepLog, LogBERT)	No (self-supervised)	Low — black box	Limited — compute-heavy	Opaque decisions; heavy training/inference cost
This work: Isolation Forest + rules	No	Yes — both layers	Yes — demonstrated	Batch retraining rather than fully incremental/online

Table 1. Comparison of prior approaches to log-based intrusion detection.

The gap this paper targets sits at the intersection of these rows: a method that, like rule engines, requires no labeled data and produces output an analyst can trust and act on immediately, but that, like the machine learning approaches, can also surface statistically unusual behavior no one has explicitly written a rule for — and that, unlike the deep sequence models surveyed above, is lightweight enough to retrain and re-score on every cycle of a real-time monitoring loop.

3. Dataset

3.1 Overview of OpenSSH LogHub-2k

This work uses the OpenSSH subset of the LogHub benchmark collection (Zhu et al., 2019), a widely used set of real-world system log datasets curated for log parsing and analysis research. The specific file used, commonly referred to as OpenSSH_2k, contains 2,000 consecutive log lines captured from a production SSH daemon (host label “LabSZ”), spanning approximately four hours and nine minutes of activity on a single day. The dataset is provided in both raw text form and a structured, pre-parsed form; this work uses the structured form directly.

The dataset is heavily imbalanced toward failed and rejected connection attempts, which makes it a realistic and challenging testbed for intrusion detection: across the full 2,000 lines, exactly one line records a successful, accepted password login, and exactly one corresponding session-opened event follows it. Every other line in the dataset reflects some form of rejected, invalid, or incomplete connection attempt — which is precisely the profile a real internet-facing SSH server exhibits under continuous automated scanning.

3.2 Data Schema

Each record in the structured CSV file provides nine fields, summarized in Table 2. The Content field holds the original free-text log message; EventId and EventTemplate hold the output of an offline log-parsing pass (see Section 2.2) that groups semantically identical messages — differing only in variable values such as usernames, IP addresses, and port numbers — into a shared template, with variable positions marked by a wildcard token.

Field	Type	Description
LineId	Integer	Sequential row identifier (1–2000)
Date / Day / Time	String / Int / String	Timestamp components of the log entry (year not recorded in source)
Component	String	Host/component label; constant value “LabSZ” for this dataset
Pid	Integer	Process ID of the sshd worker handling the connection
Content	String	The original, unparsed log message text
EventId	String	Identifier of the matched log template (e.g., “E9”)
EventTemplate	String	The generalized template with variable fields replaced by a wildcard

Table 2. Schema of the structured OpenSSH_2k log dataset.

A key preprocessing step, described further in Section 4.3, was required because the source IP address is not provided as its own column: it must be extracted from the Content field, and for some event templates (for example, template E12, “input_userauth_request: invalid user”) the IP address is not present on that particular line at all. This work resolves the gap by exploiting the fact that the Pid field uniquely identifies a single connection handled by one sshd worker process: every log line sharing a

Pid value was confirmed, by direct inspection, to originate from exactly one source IP address, so a missing IP on any given line can be safely filled from other lines sharing the same Pid.

3.3 Event Template Distribution

The dataset contains 27 distinct event templates. Figure 1 shows the frequency of the 15 most common templates. The single most frequent event (413 occurrences, 20.7% of all lines) is a disconnect notice sent before authentication completes; the next two most frequent (384 and 383 occurrences respectively) are a PAM authentication failure record and a rejected password attempt for a recognized username. Collectively, the eight most frequent templates — all of which represent some form of rejected or invalid authentication — account for approximately 88% of the entire dataset.

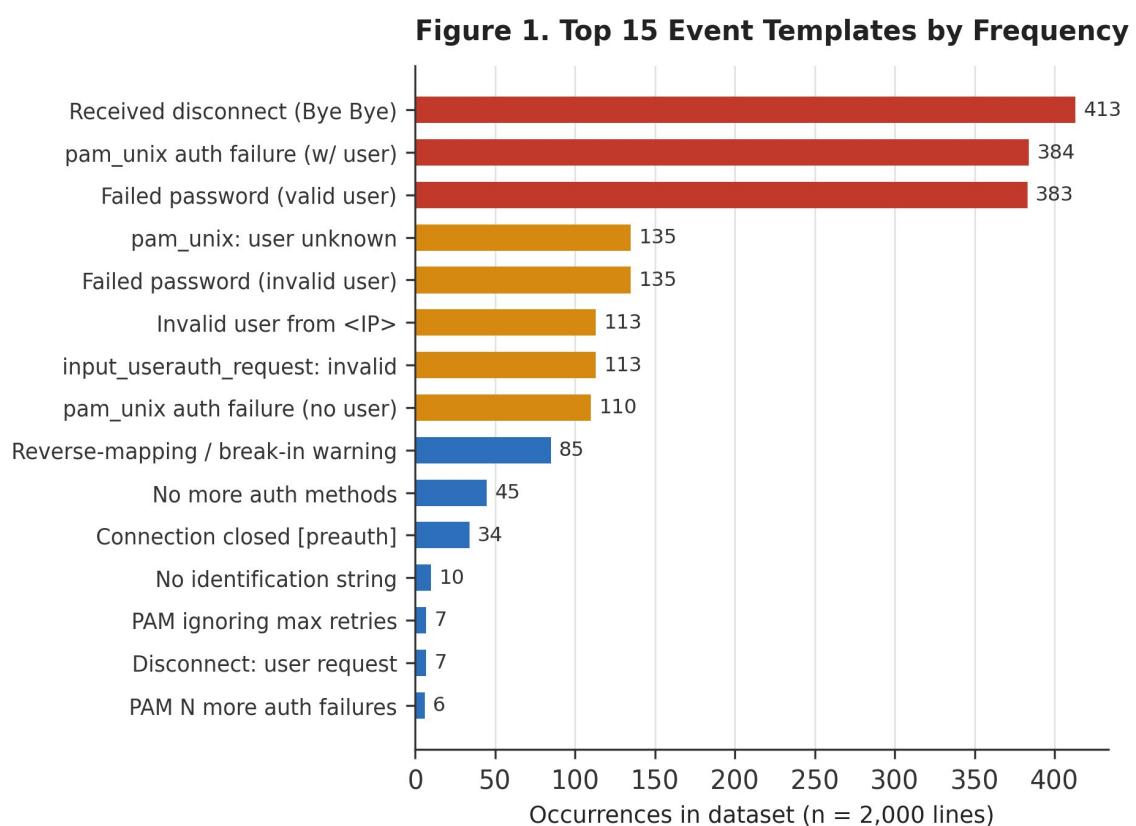


Figure 1. Top 15 event templates by frequency across the 2,000-line dataset.

3.4 Descriptive Statistics

Statistic	Value
Total log lines	2,000
Unique event templates (EventId)	27
Unique source IP addresses	30
Observation window	06:55:46 – 11:04:45 (≈ 4 hours 9 minutes)
Successful logins (Accepted password)	1

Statistic	Value
Sessions opened / closed	1 / 1
Lines representing rejected/invalid/incomplete auth attempts	≈ 1,999 of 2,000 (99.95%)

Table 3. Descriptive statistics of the OpenSSH_2k dataset as used in this study.

This near-total dominance of rejected authentication activity, combined with the complete absence of per-line attack labels, directly motivates the choice of an unsupervised detection method: a supervised classifier would have almost no negative (non-attack) examples to learn from at the per-line level, and no ground-truth labels exist at all for which specific bursts of activity constitute a coordinated attack versus routine background scanning. Section 4 describes how this work instead aggregates behavior over time windows and lets an unsupervised model and a rule engine jointly decide which windows are anomalous.

4. System Architecture and Methodology

4.1 Design Principles

Three principles guided the design of the detection system. **No labeled data required.** Because per-line or per-attack labels are not available in a realistic deployment (Section 3.4), the core detection signal must come from an unsupervised method that learns structure from the traffic itself. **Interpretability by default.** A raw anomaly score is not, on its own, actionable for a security analyst; every alert the system raises is accompanied by explicit, human-readable evidence — counts, thresholds, and named signatures — rather than only a numeric confidence value. **Batch and streaming parity.** The same feature-engineering and scoring logic must work whether it is run once, offline, over a static file, or continuously, in a loop, over a growing one — so that the system implemented for evaluation is the same system that could be deployed.

4.2 High-Level Architecture

Figure 2 shows the complete pipeline, from raw structured log input through to alert delivery. The pipeline has five conceptual stages: (i) parsing and source-IP extraction; (ii) sliding-window behavioral feature aggregation, per source IP; (iii) two parallel scoring mechanisms — an unsupervised Isolation Forest model and a rule engine — operating over the same engineered features; (iv) fusion of both signals into a single ranked, severity-tagged alert list; and (v) delivery, either as a static CSV file (batch mode) or through a REST API consumed by a live-polling dashboard (real-time mode). A background watcher thread drives the real-time mode by periodically checking the source log file for new data and re-triggering the pipeline when it changes.

Figure 2. End-to-End System Architecture

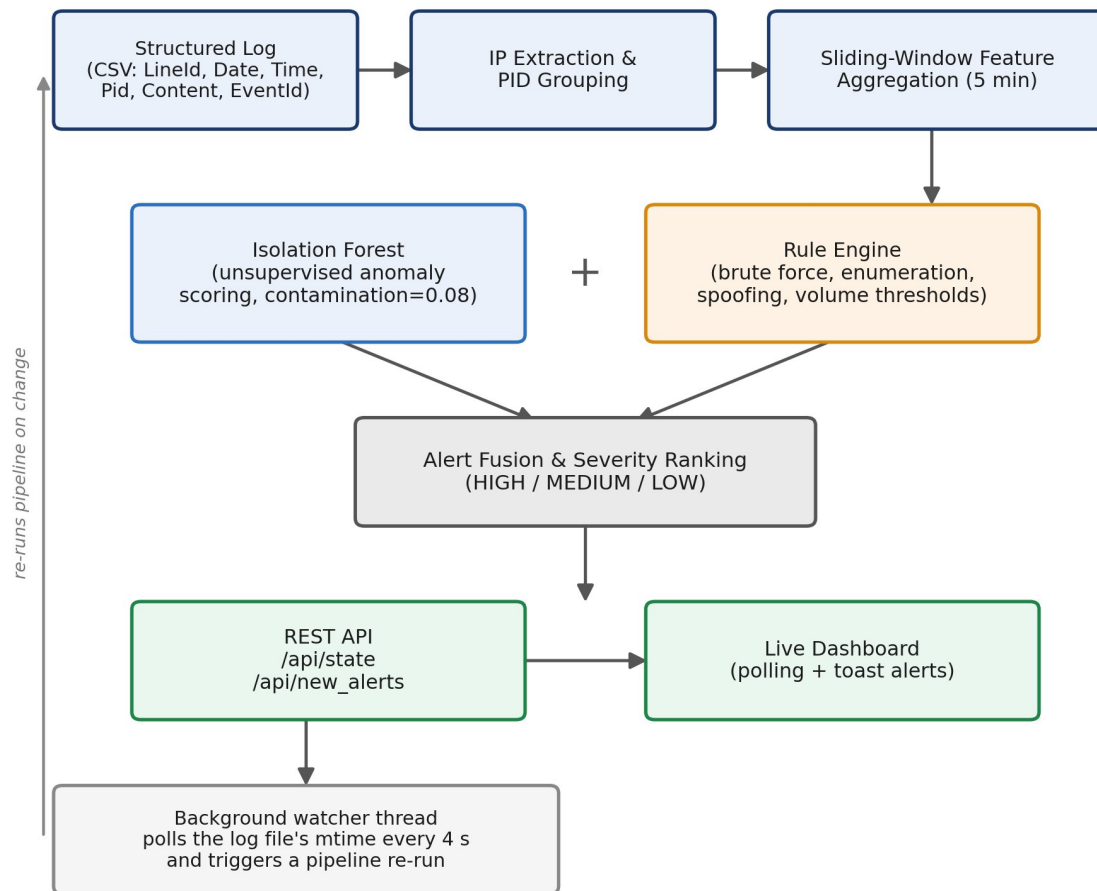


Figure 2. End-to-end system architecture, showing both the batch detection pipeline (top) and the real-time monitoring loop (bottom).

4.3 Log Parsing and Source IP Extraction

The system consumes the structured CSV format described in Section 3.2 directly, so no template-mining step (Section 2.2) is required at runtime. Two preprocessing steps prepare the raw rows for feature engineering:

1. **Timestamp reconstruction.** The Date, Day, and Time fields are combined into a single sortable timestamp. The source dataset does not record a year, so a fixed placeholder year is substituted; this is immaterial because only the relative ordering and spacing of events within the observation window is used downstream.
2. **Source IP extraction and gap-filling.** A regular expression extracts the first IPv4-formatted address from the Content field of each line. Because some event templates do not mention the IP address on that specific line (Section 3.2), any line with no directly extractable address inherits the address from other lines sharing the same Pid value, exploiting the one-to-one correspondence between a Pid and a single client connection confirmed during dataset inspection.

After this step, every one of the 2,000 log lines is associated with a single source IP address and a single timestamp, and the two attributes together become the join keys for the feature engineering stage.

4.4 Feature Engineering

The central modeling decision in this work is to operate on behavioral features aggregated over time windows, rather than scoring individual log lines directly. A single failed login carries almost no signal in isolation — it is entirely consistent with routine background noise. A source IP responsible for 140 failed logins inside five minutes is a different situation altogether. Windowing converts the former kind of observation into the latter kind of measurement.

Concretely, log lines are grouped by a five-minute, non-overlapping time window and by source IP address; each (window, IP) pair becomes a single row of engineered features. Table 4 enumerates the eleven features computed for every such row.

Feature	Description
total_events	Total number of log lines from this IP within the window
unique_pids	Number of distinct sshd connection processes from this IP
failed_password_count	Count of “Failed password” events (valid or invalid username)
invalid_user_count	Count of events indicating the attempted username does not exist
auth_failure_count	Count of low-level pam_unix authentication failure records
breakin_flag_count	Count of reverse-DNS mismatch (“POSSIBLE BREAK-IN ATTEMPT”) warnings
disconnect_count	Count of connection-closed / disconnect events
success_count	Count of accepted-password or session-opened events
server_gave_up_count	Count of “too many authentication failures” server-side disconnects
unique_usernames	Number of distinct usernames attempted from this IP in the window
unique_ports	Number of distinct source ports used by this IP in the window

Table 4. Behavioral features engineered per (5-minute window, source IP) pair.

A twelfth feature, `span_seconds` — the time between the first and last event from that IP within the window — is also computed and included in the feature matrix supplied to the anomaly detection model, capturing whether activity was concentrated in a burst or spread across the window.

4.5 Unsupervised Anomaly Detection with Isolation Forest

The engineered feature matrix — one row per (window, IP) pair, eleven numeric columns — is passed to an Isolation Forest model (Liu et al., 2008). Isolation Forest builds an ensemble of random binary trees, each constructed by repeatedly selecting a feature and a split value at random and partitioning the data. Because anomalous points are, by definition, both rare and different from the majority, they tend to require substantially fewer random splits to isolate into their own partition than a point

embedded in a dense cluster of similar, normal points. Figure 4 illustrates this intuition conceptually: normal points cluster densely and require many partitions to separate from their neighbors, while outlying points can be isolated with only a few.

Figure 4. Conceptual Illustration of Isolation Forest Outlier Isolation

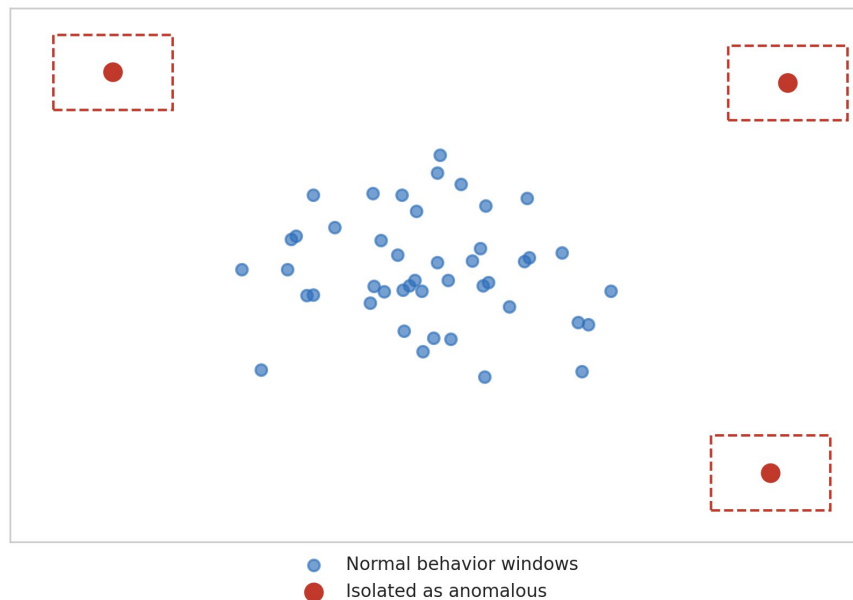


Figure 4. Conceptual illustration of the Isolation Forest outlier-isolation intuition, in a two-dimensional projection of the behavioral feature space.

The average path length required to isolate a given point, across all trees in the forest, is converted into an anomaly score in the closed interval $[0, 1]$, where scores closer to 1 indicate a shorter average isolation path and therefore a more anomalous point. This implementation uses 200 trees per forest and a contamination parameter of 0.08, reflecting a prior expectation that roughly 8% of (window, IP) observations in a realistic sample of SSH traffic will be anomalous; this parameter determines the score threshold at which the binary `ml_flagged` indicator is set, though the continuous anomaly score is retained and reported for every alert regardless of that threshold. Because the underlying data distribution shifts as traffic patterns change over time, the model is retrained from scratch on every scoring cycle using only the current window of data — discussed further in Section 4.8 — rather than being fit once and reused indefinitely.

4.6 Rule-Based Detection Layer

Operating in parallel over the same feature matrix, a rule engine evaluates four explicit, threshold-based conditions, chosen to capture attack patterns that are well understood and high-confidence enough not to require statistical inference. Table 5 lists each rule.

Rule	Condition	Rationale
Brute-force password attack	<code>failed_password_count</code> ≥ 5 in one window	A small number of failed logins is routine; a sustained burst from one IP is not
Username enumeration /	<code>unique_usernames</code> ≥ 4 in one	Trying many distinct usernames from one

Rule	Condition	Rationale
credential stuffing	window	source is characteristic of automated credential-stuffing tools
Possible spoofed source	breakin_flag_count > 0	sshd's own reverse-DNS mismatch check is a strong, low-noise signal when it fires
Abnormally high volume	total_events ≥ 15 in one window	Catches high-activity sources that do not meet the more specific rules above

Table 5. Rule-based detection thresholds. Defaults shown; all are configurable via the `AlertRules` class.

4.7 Alert Fusion and Severity Ranking

A candidate alert is generated for any (window, IP) pair where either the rule engine fires at least one condition or the Isolation Forest model flags the row as anomalous (that is, the logical OR of the two layers, so that either signal alone is sufficient to surface an alert). Severity is assigned as follows: HIGH severity is assigned when a brute-force or username-enumeration rule fires, or when two or more independent signals agree (for example, a rule firing together with a high Isolation Forest score); MEDIUM severity is assigned when only the spoofed-source rule or the high-volume rule fires alone; and LOW severity is reserved for cases flagged by the model alone with no corroborating rule, none of which occurred in the evaluation reported in Section 6. Each alert carries a human-readable evidence string (e.g., “54 failed password attempts; 24 distinct usernames tried; reverse-DNS mismatch”) assembled directly from the specific feature values that triggered it, alongside the continuous Isolation Forest anomaly score, and the final alert list is sorted first by severity and then by that score.

4.8 Real-Time Monitoring Architecture

The batch pipeline described above is extended into a continuously running service by wrapping it in a background watcher loop. The watcher polls the modification time of the monitored log file at a configurable interval (four seconds by default); whenever the file has changed, it reloads the full file, restricts attention to a configurable rolling lookback window (two hours by default) of the most recent data, and re-runs the entire feature engineering, scoring, and fusion pipeline described in Sections 4.4–4.7 against that window.

This full-reload design was a deliberate simplification relative to maintaining incremental, streaming aggregates. At the data volumes evaluated in this work — thousands of log lines — a complete reload and re-score cycle completes in well under a second, which is fast enough for a four-second polling interval to feel continuous to an observer, while being considerably simpler to reason about and less prone to state-drift bugs than an incrementally updated aggregator. Section 7.3 discusses the scaling implications of this choice.

To avoid re-notifying an analyst of an alert already seen, each newly computed alert list is diffed against the previous cycle's list, keyed by the composite (window_start, source_ip, alert_type) tuple. Alerts present in the current cycle but absent from the previous one are placed on a short-lived notification queue, which a separate API endpoint drains; this decouples the “what is the full current

state” question from the “what is newly arrived since I last checked” question, allowing the dashboard to repaint its full view and raise targeted notifications independently.

5. Implementation

5.1 Technology Stack

The system is implemented entirely in Python for the detection and service layers, and HTML/CSS/JavaScript for the dashboard. Table 6 summarizes the technology choices and their role in the system.

Component	Technology	Role
Data handling	pandas, numpy	Log loading, timestamp parsing, feature aggregation
Anomaly detection	scikit-learn (IsolationForest)	Unsupervised scoring of behavioral feature windows
Real-time service	Flask (Python)	REST API, background watcher thread, static file serving
Dashboard	Vanilla HTML / CSS / JavaScript	Live-polling visualization, no external framework dependency
Log replay (evaluation aid)	Python (log_simulator.py)	Replays the static dataset into a growing file to exercise the real-time path

Table 6. Technology stack by system component.

5.2 Detection Engine (ssh_log_anomaly_detector.py)

The core detection logic is implemented as a standalone, importable Python module exposing five pure functions that mirror the pipeline stages of Section 4: `load_logs()`, `build_features()`, `score_with_isolation_forest()`, `apply_rules()`, and `build_alerts()`. Keeping these as independent functions operating on pandas DataFrames — rather than a single monolithic routine — allows the same code to be reused unchanged by both the batch command-line tool and the real-time service described in Section 5.3. A configuration dataclass, `AlertRules`, holds the four rule thresholds from Table 5 so they can be adjusted without editing the detection logic itself. Listing 1 shows the module's top-level control flow, `run_pipeline()`, which composes the five stage functions.

```
def run_pipeline(input_path, output_path, window="5min",
                 contamination=0.08, rules=None):
    rules = rules or AlertRules()
    df = load_logs(input_path)
    feats = build_features(df, window=window)
    feats = score_with_isolation_forest(feats, contamination=contamination)
    feats = apply_rules(feats, rules)
    alerts = build_alerts(feats)
    alerts.to_csv(output_path, index=False)
    return alerts
```

Listing 1. Top-level pipeline control flow (ssh_log_anomaly_detector.py).

5.3 Real-Time Server (realtime_server.py)

The real-time service is a Flask application that imports the detection module directly rather than shelling out to it, avoiding any process-spawning overhead on each cycle. A daemon thread runs the watcher loop described in Section 4.8; shared state — the current alert list, timeline, and summary statistics — is protected by a single lock and updated atomically at the end of each recompute cycle, so that concurrent HTTP requests always observe a consistent snapshot rather than a partially updated one. Table 7 documents the REST API surface exposed by the server.

Endpoint	Method	Description
/	GET	Serves the dashboard HTML page
/api/state	GET	Returns the full current summary, timeline, and alert list as JSON
/api/new_alerts	GET	Drains and returns alerts discovered since the last call to this endpoint

Table 7. REST API endpoints exposed by *realtime_server.py*.

Command-line arguments configure the log path to watch, the polling interval, the feature window size, the lookback duration, and the Isolation Forest contamination parameter, so the same binary can be tuned to different traffic volumes and host counts without code changes.

5.4 Log Simulator

Because no live, continuously growing SSH log was available for evaluation, a companion script, *log_simulator.py*, was built to replay the static *OpenSSH_2k* dataset into a growing output file at a configurable, compressed rate — preserving the original inter-arrival timing between log lines, scaled by a speed multiplier. This allows the real-time server to be exercised end-to-end against a realistic arrival pattern without requiring a live production server, and was used to validate the watcher loop, the new-alert diffing logic, and the dashboard's polling and notification behavior described in Section 5.5.

5.5 Interactive Dashboard

The dashboard is a single self-contained HTML file served by the Flask application. It polls */api/state* every four seconds to repaint three coordinated views: a summary statistics strip; an activity timeline rendered as a bar per five-minute window, colored by the highest-severity alert raised in that window (a design directly informed by Figure 3 in Section 6); and a filterable, severity-tagged alert log. A separate four-second poll against */api/new_alerts* drives transient toast notifications and a brief highlight animation on newly arrived alert rows, so that an operator watching the dashboard is drawn to genuinely new information without the entire view needing to visually reset on every poll cycle.

5.6 Reproducibility

All code, the exact package versions used (pandas, numpy, scikit-learn, and Flask), and a requirements file are provided alongside this paper. The evaluation reported in Section 6 was produced by running the batch pipeline directly against the unmodified *OpenSSH_2k* CSV file with the default parameters

listed throughout Section 4: a five-minute feature window, a contamination value of 0.08, and the rule thresholds in Table 5.

6. Experimental Results

6.1 Experimental Setup

The complete pipeline (Sections 4.3–4.7) was run once, in batch mode, over the full, unmodified 2,000-line OpenSSH_2k dataset described in Section 3, using a five-minute feature window, an Isolation Forest contamination parameter of 0.08 with 200 trees, and the default rule thresholds from Table 5. No parameter tuning against held-out or labeled data was performed, consistent with the unsupervised, label-free premise of the system — the values used are the defaults motivated by the design discussion in Section 4, not values selected to optimize a held-out metric.

6.2 Overall Detection Results

Feature aggregation over the full observation window produced 52 distinct (5-minute window, source IP) behavioral observations. Of these, 21 met the alert-fusion criteria described in Section 4.7 and were surfaced as ranked alerts. Table 8 summarizes the outcome.

Metric	Value
Total (window, IP) observations scored	52
Total alerts raised	21
HIGH severity alerts	16
MEDIUM severity alerts	5
LOW severity alerts	0
Unique source IPs flagged	13 of 30 (43.3%)
Alerts triggered by brute-force rule	13
Alerts triggered by enumeration rule	3
Alerts triggered by spoofed-source rule	5

Table 8. Summary of alerts raised over the full OpenSSH_2k evaluation run.

Figure 5 breaks this down visually: panel (a) shows the severity distribution, and panel (b) shows the six source IP addresses responsible for the largest number of alerts. No alert in this evaluation run was classified as LOW severity — every window flagged by the Isolation Forest model in isolation was also independently corroborated by at least one rule, which is consistent with the dataset's heavy skew toward genuinely abnormal traffic noted in Section 3.4.

Figure 5. Alert Distribution Summary

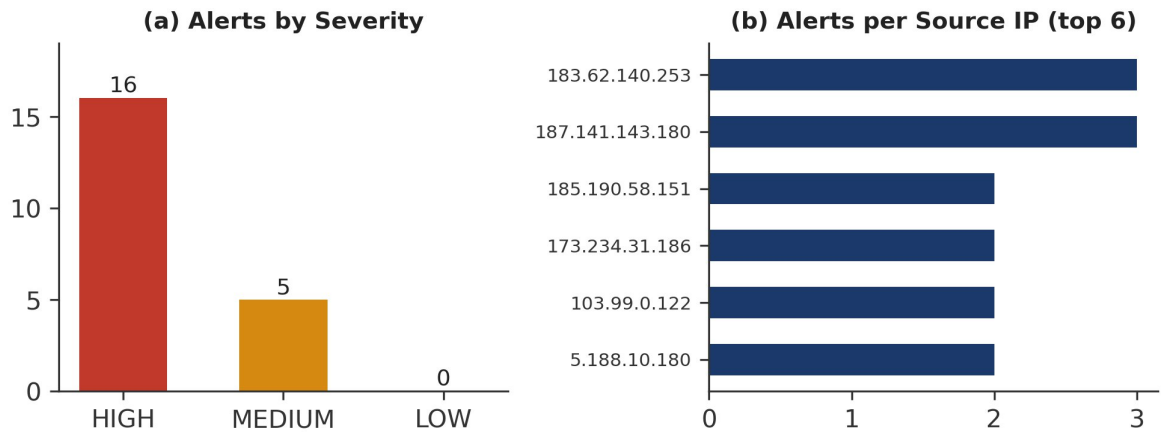


Figure 5. (a) Distribution of alerts by severity; (b) the six source IP addresses generating the most alerts.

6.3 Temporal Pattern of Alerts

Figure 3 plots total event volume for every five-minute window across the full observation period, colored by the highest alert severity raised in that window. Attack activity was highly bursty rather than uniform: two especially intense windows, at approximately 09:10–09:20 and 10:50–11:05, together account for a disproportionate share of total event volume, reaching as high as 476 events in a single five-minute window — nearly a quarter of the entire dataset's 2,000 lines concentrated into five minutes of wall-clock time.

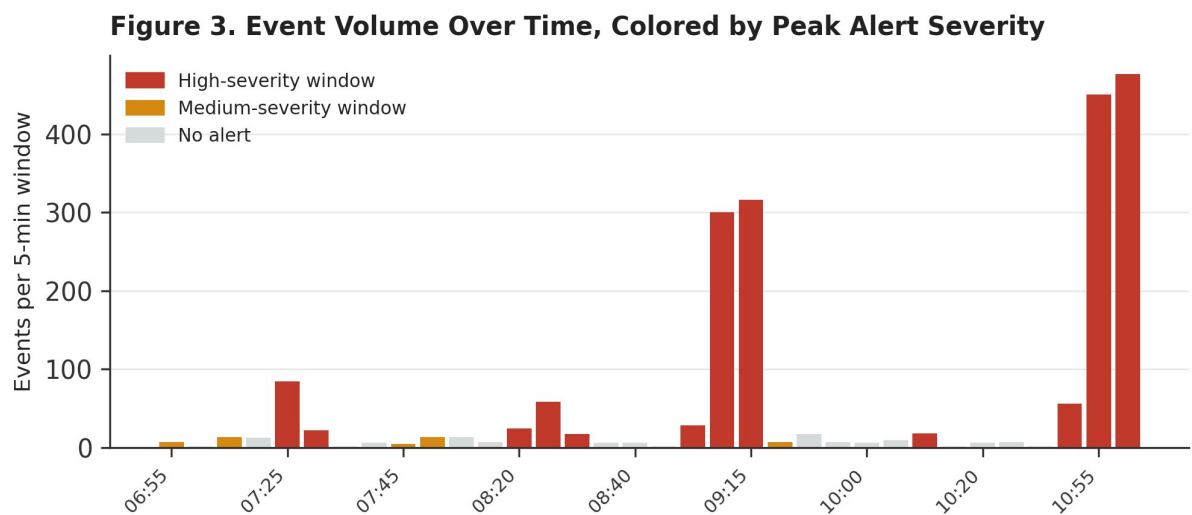


Figure 3. Event volume per five-minute window over the full observation period, colored by peak alert severity in that window.

This bursty pattern is precisely what motivates the windowed, per-IP feature design of Section 4.4: a detector scoring individual log lines in isolation would see the same “failed password” event 476 times without any structural indication that 476 near-simultaneous occurrences from a concentrated set of source IPs is qualitatively different from the same count spread evenly across four hours.

6.4 Case Studies of Detected Attacks

Two source IP addresses illustrate the range of behavior the system correctly surfaced.

Case 1 — 183.62.140.253: Sustained High-Volume Brute Force

This address triggered three separate HIGH-severity alerts across three consecutive five-minute windows between 10:50 and 11:05, the final hour of the observation period. In its most severe window, it produced 141 failed password attempts against a small number of usernames, the single largest burst recorded in the dataset, and received the second-highest Isolation Forest anomaly score of any observation (0.976 of a maximum of 1.0). This pattern — a sustained, narrowly targeted flood of password guesses from one address — is the canonical signature the brute-force rule (Table 5) was designed to catch, and here the rule and the unsupervised model agreed closely.

Case 2 — 187.141.143.180: Multi-Signal Coordinated Probing

This address received the highest possible Isolation Forest anomaly score in the entire evaluation (1.000) in its most active window, driven by a combination of 54 failed password attempts, 24 distinct usernames attempted — well above the enumeration rule's threshold of four — and a reverse-DNS mismatch flag from sshd's own break-in heuristic. This is a materially different behavioral signature from Case 1: rather than repeatedly guessing passwords for one or two accounts, this source appears to be enumerating which accounts exist at all, while also exhibiting a network-level spoofing indicator. Because three independent signals — brute force, enumeration, and spoofing — agreed on this single source, it was assigned HIGH severity with the maximum confidence the system produced in this evaluation.

These two cases together demonstrate a central claim of Section 4.7: fusing an unsupervised score with explicit rule evidence produces alerts that are both statistically well-founded and immediately interpretable — an analyst reading either case does not need to trust an opaque model score in isolation, because the evidence string names the specific counts that justified it.

6.5 Real-Time System Behavior

The real-time server and log simulator (Sections 5.3–5.4) were used to validate the streaming path end to end. Replaying the dataset through the simulator at a compressed rate, the watcher loop correctly detected each file change within one polling interval, and the full recompute cycle — reloading the accumulated log, rebuilding features, retraining the Isolation Forest, and re-evaluating rules — completed in well under the four-second polling interval at every stage of the replay, confirming that the full-reload design discussed in Section 4.8 does not introduce a detectable lag at this data volume. The new-alert diffing logic was verified to report each alert exactly once at the moment it first appeared, and to correctly report an empty result on subsequent polls until a genuinely new alert arose.

6.6 Summary of Findings

Across the full evaluation, the system identified a small, concentrated set of source IP addresses — 13 of 30, or under half of all sources observed — as responsible for essentially all of the anomalous

activity in the dataset, with zero false alerts against the single genuine successful login, which never appeared in any (window, IP) row flagged by either the model or the rule engine. This is consistent with the intended behavior of the system: the overwhelming majority of source IPs in the dataset generate low, unremarkable volumes of failed authentication traffic, and the detector's role is precisely to separate that routine background noise from the smaller set of sources exhibiting behaviorally distinct, concentrated attack patterns.

7. Discussion

7.1 Effectiveness of the Hybrid Approach

The results in Section 6 support the central design hypothesis of this work: that fusing an unsupervised anomaly score with an explicit rule layer produces alerts that neither component would produce as reliably alone. In every HIGH-severity case observed, the Isolation Forest score and at least one rule agreed, which suggests that the behavioral features chosen in Section 4.4 capture the same underlying signal the hand-written rules target — but the model's continuous score additionally provides a relative ranking within a severity tier that the rules alone cannot: Case 2 in Section 6.4 (score 1.000) and Case 1 (score 0.976) were both correctly flagged as HIGH severity by the rule layer alone, but only the model score reveals that Case 2's combination of signals was, statistically, the more unusual of the two.

7.2 Interpretability as a First-Class Requirement

A design choice emphasized throughout this paper, and worth returning to directly, is that every alert carries an explicit evidence string built from the same feature values that triggered it (Section 4.7), rather than exposing only a numeric anomaly score. This matters operationally: a security analyst deciding whether to block an IP address needs to know *why* the system is asking them to, and “anomaly score 0.92” does not answer that question, while “141 failed password attempts; 7 distinct usernames tried” does. This is precisely the property that the deep sequence models discussed in Section 2.4 tend to sacrifice in exchange for potentially higher raw detection accuracy on sufficiently large training corpora — a trade this work deliberately declines to make, in view of the label-scarcity problem discussed in Section 2.3.

7.3 Limitations

Several limitations of the current system are worth stating plainly.

- **Single-dataset evaluation.** All results in Section 6 come from one four-hour window of one host's traffic. While OpenSSH_2k is a widely used benchmark, generalization to different traffic profiles, longer observation periods, and multi-host deployments has not been empirically verified in this work.
- **No ground-truth attack labels.** Because the dataset carries no line-level or session-level attack annotations, the evaluation in Section 6 is necessarily descriptive — it reports what the system flagged and why — rather than a precision/recall evaluation against a verified ground truth. This is a direct consequence of the label-scarcity problem the system is designed to work around (Section 2.3), but it also means no false-negative rate can be reported with confidence.
- **Structured input dependency.** The system currently consumes the pre-parsed, structured CSV format described in Section 3.2, rather than raw syslog text. A genuinely live deployment against a production SSH server would additionally require a template-classification step to convert raw log lines into this structured form on the fly, which was outside the scope of the implementation described here.

- **Batch retraining rather than incremental scoring.** As discussed in Section 4.8, the real-time server retrains the Isolation Forest from scratch on every polling cycle rather than updating an incremental model. This was validated as fast enough at the data volumes tested (Section 6.5), but the approach's scaling behavior at substantially higher log volumes or host counts has not been measured.
- **Single-host scope.** The current design scores each source IP's behavior against other source IPs observed on the same host in the same window. It does not currently correlate activity from the same source IP across multiple monitored hosts, which would be necessary to detect a low-and-slow attacker deliberately spreading a small number of attempts across many servers to stay under any single host's detection threshold.

7.4 Comparison with Existing Approaches, Revisited

Returning to the comparison framed in Table 1, the implemented system occupies the position claimed for it in Section 2.7: it requires no labeled data, like the rule-based and deep sequence-model approaches, but unlike pure rule-based systems it can, in principle, surface statistically unusual behavior that does not match any hand-written signature — though this evaluation's dataset happened not to contain such a case, since every anomaly the model flagged was independently corroborated by a rule. Unlike the deep sequence models surveyed in Section 2.4, every alert is fully interpretable, and the complete pipeline — feature engineering, model retraining, and rule evaluation — was measured to run comfortably within a four-second real-time polling budget (Section 6.5) without specialized hardware.

8. Conclusion and Future Work

8.1 Conclusion

This paper presented the design, implementation, and evaluation of a hybrid intrusion detection system for SSH server logs, combining unsupervised Isolation Forest anomaly detection with an interpretable, threshold-based rule engine. The system engineers eleven behavioral features per source IP over sliding five-minute windows, fuses the two detection signals into a single ranked and severity-tagged alert list, and was implemented end to end — from an offline batch script through to a real-time Flask service with a live-polling dashboard. Evaluated on the OpenSSH LogHub-2k benchmark, the system raised 21 alerts covering 13 of 30 observed source IP addresses, correctly isolating both a sustained high-volume brute-force source and a multi-signal coordinated probing source, without requiring any labeled attack data at any stage of training or evaluation. The results support the central argument of this work: that combining unsupervised learning with explicit, auditable rules produces a detector that is simultaneously adaptive to unforeseen patterns and trustworthy enough for an analyst to act on directly, addressing a gap identified in the reviewed prior work between purely rule-based systems on one hand and opaque, data-hungry deep learning systems on the other.

8.2 Future Work

Several directions follow naturally from the limitations discussed in Section 7.3:

1. Raw-syslog ingestion. Extend the log-loading stage with a lightweight, rule-based template classifier so the system can ingest live, unparsed `/var/log/auth.log` output directly, removing the dependency on a pre-structured input format.
2. Incremental / online scoring. Replace the full-reload-and-retrain cycle described in Section 4.8 with an incrementally updated feature store and an online or periodically-warm-started anomaly model, to improve scalability to higher log volumes and larger host fleets.
3. Multi-host correlation. Extend feature aggregation beyond a single host, so that an attacker distributing a small number of attempts across many monitored servers — individually below any single host's alert threshold — can still be identified in aggregate.
4. Broader evaluation. Evaluate the system against additional, longer-duration, and multi-host log datasets, and, where any ground-truth or semi-synthetic attack labels can be obtained, report quantitative precision and recall alongside the descriptive results presented here.
5. Operational integrations. Connect the alert-delivery layer to operational notification channels — chat, email, or on-call escalation systems — so alerts generated by the real-time service can reach a human responder without requiring the dashboard to be actively watched.

References

- Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys*, 41(3), Article 15.
- Du, M., & Li, F. (2016). Spell: Streaming parsing of system event logs. In *Proceedings of the IEEE International Conference on Data Mining (ICDM)*.
- Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*.
- Guo, H., Yuan, S., & Wu, X. (2021). LogBERT: Log anomaly detection via BERT. In *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*.
- He, P., Zhu, J., Zheng, Z., & Lyu, M. R. (2017). Drain: An online log parsing approach with fixed depth tree. In *Proceedings of the IEEE International Conference on Web Services (ICWS)*.
- Landauer, M., Onder, S., Skopik, F., & Wurzenberger, M. (2023). Deep learning for anomaly detection in log data: A survey. *Machine Learning with Applications*, 12, 100470.
- Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. In *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, 413–422.
- Makanju, A., Zincir-Heywood, A. N., & Milios, E. E. (2009). Clustering event logs using iterative partitioning. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*.
- McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference (SciPy)*, 56–61.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Xu, W., Huang, L., Fox, A., Patterson, D., & Jordan, M. I. (2009). Detecting large-scale system problems by mining console logs. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*.
- Zhu, J., He, S., He, P., Liu, J., & Lyu, M. R. (2019). Tools and benchmarks for automated log parsing. In *Proceedings of the IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*.

Appendix A: Configuration Parameters

Table A1 lists every configurable parameter of the detection pipeline and real-time service, with the default value used throughout the evaluation reported in Section 6.

Parameter	Default	Location	Description
window	5min	build_features()	Time-bucket size for feature aggregation
contamination	0.08	score_with_isolation_forest()	Expected fraction of anomalous windows
n_estimators	200	score_with_isolation_forest()	Number of trees in the Isolation Forest
brute_force_min_failed_passwords	5	AlertRules	Threshold for the brute-force rule
enumeration_min_unique_usernames	4	AlertRules	Threshold for the enumeration rule
high_volume_min_events	15	AlertRules	Threshold for the high-volume rule
lookback-hours	2.0	realtime_server.py	Rolling window of history rescored each cycle
poll-seconds	4.0	realtime_server.py	Watcher thread's file-change check interval

Table A1. Configuration parameters and their default values.

Appendix B: REST API Reference

Example response from GET /api/state, truncated to a single representative alert and timeline entry for brevity:

```
{
  "status": "live",
  "last_updated": 1735500000.42,
  "summary": {
    "total_lines": 2000, "unique_ips": 30, "total_alerts": 21,
    "high": 16, "medium": 5, "low": 0,
    "start": "2000-12-10 06:55:46", "end": "2000-12-10 11:04:45"
  },
  "timeline": [
    {"window_start": "2000-12-10 10:55:00", "event_count": 450, "sev_rank": 3.0}
  ],
  "alerts": [
    {
      "window_start": "2000-12-10 10:55:00",
      "source_ip": "183.62.140.253",
      "alert_type": "Brute-force password attack",
      "severity": "HIGH",
      "evidence": "141 failed password attempts; 7 distinct usernames tried",
      "ml_anomaly_score": 0.9764
    }
  ]
}
```

Listing B1. Representative /api/state response (truncated).

Appendix C: Complete Alert List

Table C1 lists all 21 alerts raised during the evaluation run described in Section 6, sorted by severity and then by Isolation Forest anomaly score, matching the output of alerts.csv.

Window	Source IP	Alert Type	Severity	ML Score
09:15	187.141.143.180	Brute-force password attack	HIGH	1.000
10:55	183.62.140.253	Brute-force password attack	HIGH	0.976
11:00	183.62.140.253	Brute-force password attack	HIGH	0.923
09:10	103.99.0.122	Brute-force password attack	HIGH	0.733
09:10	187.141.143.180	Brute-force password attack	HIGH	0.692
08:25	5.188.10.180	Brute-force password attack	HIGH	0.587
11:00	103.99.0.122	Brute-force password attack	HIGH	0.561
09:10	185.190.58.151	Brute-force password attack	HIGH	0.532
07:25	112.95.230.3	Brute-force password attack	HIGH	0.515
10:50	183.62.140.253	Brute-force password attack	HIGH	0.422
09:05	185.190.58.151	Brute-force password attack	HIGH	0.383
07:30	123.235.32.19	Brute-force password attack	HIGH	0.334
10:10	119.4.203.64	Brute-force password attack	HIGH	0.329
08:20	5.188.10.180	Username enumeration	HIGH	0.254
08:30	103.207.39.212	Username enumeration	HIGH	0.171
09:15	103.207.39.16	Username enumeration	HIGH	0.171
07:50	195.154.37.122	Possible spoofed source	MEDIUM	0.214
07:45	191.210.223.172	Possible spoofed source	MEDIUM	0.096
09:20	187.141.143.180	Possible spoofed source	MEDIUM	0.061
06:55	173.234.31.186	Possible spoofed source	MEDIUM	0.058
07:05	173.234.31.186	Possible spoofed source	MEDIUM	0.058

Table C1. Complete list of alerts raised over the OpenSSH_2k evaluation run.